

Introduction à l'algorithmique et la programmation

Dr Ibrahima GAYE - Université Alioune DIOP de Bambey

5 avril 2021

CHAPITRE I : Introduction Générale

Table des matières

1	Introduction	3
1.1	Qu'est-ce-que c'est l'algorithmique ?	3
1.1.1	Histoire	3
1.1.2	Définition	3
1.1.3	Alors, l'algorithmique, c'est quoi ?	3
1.1.4	Exemples d'Algorithmes Elémentaires	4
1.2	Programmation	5
1.3	Compilation et exécution	5

1 Introduction

Le but de ce cours est d'apprendre à réfléchir d'une manière logique.

1.1 Qu'est-ce-que c'est l'algorithmique ?

1.1.1 Histoire

Origine du terme ? :

Al-Kharezmi, auteur du traité "Kitab al jabr w'al-muqabala", est l'inventeur des manipulations algébriques (**algèbre = al jabr**). C'est Léonard de Pise, dit Fibonacci, qui emprunta le nom du célèbre mathématicien arabe du 9-ième siècle pour nommer une suite d'opérations constituant un schéma de calcul ou de résolution d'un problème. Il s'agit bien entendu de l'algorithmique.

1.1.2 Définition

*Un algorithme est une suite **finie** de règles à appliquer dans un ordre déterminé à un nombre **fini** de données pour arriver, en un nombre **fini** d'étapes, à un certain résultat, et cela indépendamment des données.*

"Encyclopedia Universalis"

1.1.3 Alors, l'algorithmique, c'est quoi ?

Chercher une solution efficace et fiable à un problème donné !

Plusieurs manières de résoudre un problème, plus ou moins rapides. Le but est de trouver la plus rapide !

Pour résoudre un problème à l'aide d'un ordinateur, il faut :

1. **Analyser le problème**¹ : définir avec précision les résultats à obtenir, les informations dont on dispose.
2. **Déterminer les méthodes de résolution** : il s'agit de déterminer la suite des opérations à effectuer pour obtenir, à partir des données, la solution au problème posé. Cette suite d'opérations constitue un algorithme.

1. "The two golden rules of programming : 1-Think about the problem. 2-Think about the problem again."
[Krisboul98]

Cela repose souvent sur la décomposition un problème complexe en plusieurs problèmes plus simples. Parmi tous les algorithmes fournissant la solution, il faudra choisir le plus efficace.

3. **Formuler l'algorithme définition** : cette étape doit faciliter la résolution sur ordinateur par l'expression de l'algorithme dans un formalisme adéquat.
4. Traduire l'algorithme dans un langage de programmation adapté comme langage C, Pascal, Java, etc.

L'algorithmique se pose donc comme la base de la programmation !

Les "bons" programmes : dont il est facile de vérifier qu'ils sont corrects, qui peuvent être lus, compris, modifiés sans difficultés et qui reflètent la structure de l'algorithme de résolution.

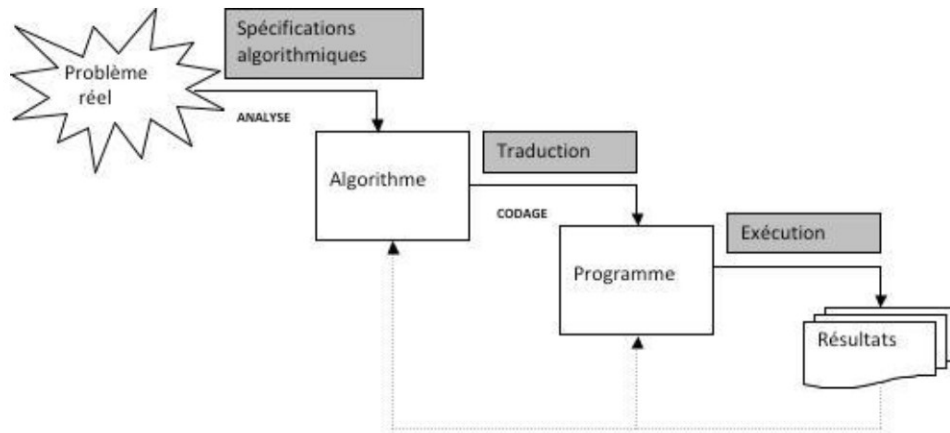


FIGURE 1 – Etape du processus de la programmation

1.1.4 Exemples d'Algorithmes Élémentaires

Les exemples suivants montrent que beaucoup d'algorithmes décrivent des tâches quotidiennes. Pour la présentation de ces algorithmes nous utiliserons tout simplement des instructions rédigées en langue française. Ces instructions seront suffisamment claires pour permettre la compréhension de leur intention.

Exemple 1 : Prise d'un médicament contre la toux

Algorithme : "En cas de toux, prendre, sauf avis contraire du médecin, un comprimé toutes les 4 heures, jusqu'à disparition des symptômes. Pour les enfants, un comprimé toutes les 8 heures suffit."

Exemple 2 : Tri d'un jeu de cartes suivant la couleur

Algorithme :

(1) Prendre la première carte ;

(2) La carte est-elle rouge ?

Si oui, poser la carte sur le premier tas ;

Sinon, poser la carte sur le second tas ;

(3) Reste-t-il des cartes ?

Si oui, prendre la carte suivante et continuer sous 2 ; Sinon, fin du tri.

1.2 Programmation

L'ordinateur n'est qu'une machine capable d'exécuter automatiquement une série d'opérations simples qu'on lui a demandé de faire. L'intérêt de l'ordinateur est sa capacité de manipuler rapidement et sans erreur un grand nombre d'informations : mémoriser des quantités numériques ou alphabétiques, rechercher une quantité mémorisée, comparer ou classer des informations.

Une fois l'algorithme trouvé, il faut l'implémenter sur machine pour pouvoir l'exécuter. Reste alors à choisir un langage.

Le programmeur est soumis aux contraintes imposées par la syntaxe du langage de programmation et à la nécessité de gérer les ressources correspondant à l'architecture de la machine.

1.3 Compilation et exécution

Le code binaire est le seul que la machine puisse directement comprendre et donc réellement exécuter. Tout programme écrit dans un langage évolué devra par conséquent être d'abord traduit en code-machine avant d'être exécuté.

Deux manières de produire du code exécutable :

1. **La compilation** : l'ensemble du programme est globalement traduit par un autre programme appelé compilateur et le code-machine produit est optimisé. Ce code-machine est ensuite exécuté autant de fois que l'on veut.
2. **L'interprétation** : un programme (interpréteur) décode une à une les instructions (ou expressions) du programme source, puis les exécute (les évalue) au fur et à mesure.